



MACHINE LEARNING NOTES

Linear Regression

সম্পূর্ণ বাংলা নোট — Intuition থেকে Implementation পর্যন্ত



Supervised Learning



Math + Code



Python Examples



২০২৬





Linear Regression — সম্পূর্ণ বাংলা নোট

বিষয়: Linear Regression | ক্যাটাগরি: Supervised Learning | তারিখ: ২০২৬

১. 🎯 সহজ ভাষায় পরিচিতি (Intuition)

এই concept কী?

ধরো তুমি একটি দোকান চালাও। প্রতিদিন তুমি লক্ষ্য করো:

- যেদিন গরম বেশি → আইসক্রিম বেশি বিক্রি হয়
- যেদিন গরম কম → আইসক্রিম কম বিক্রি হয়

তুমি একটি প্যাটার্ন দেখছো: **তাপমাত্রা বাড়লে → বিক্রি বাড়ে**। এই সম্পর্ক যদি সরলরেখায় (straight line) বোঝানো যায়, তাহলে সেটাই হলো **Linear Regression**!

বাস্তব জীবনের উদাহরণ

উদাহরণ	Input (X)	Output (Y)
🏠 বাড়ির দাম	বাড়ির আয়তন (sqft)	দাম (টাকা)
📈 আইসক্রিম বিক্রি	তাপমাত্রা (°C)	বিক্রির সংখ্যা
🎓 পরীক্ষার ফলাফল	পড়ার ঘণ্টা	নম্বর
👤 ওজন	উচ্চতা (cm)	ওজন (kg)

এটি কোন সমস্যা সমাধান করে?

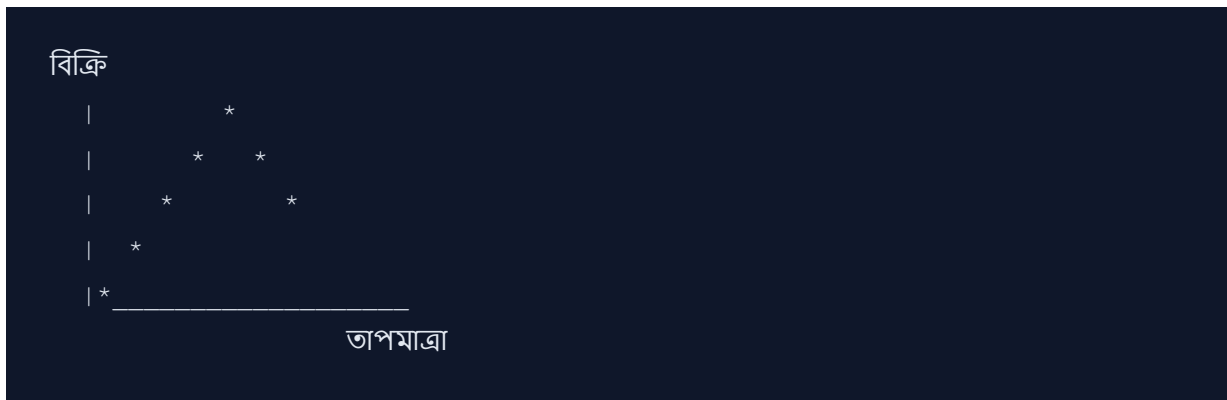
"X জানলে Y কত হবে?" — এই প্রশ্নের উত্তর দেয় Linear Regression।

- এটি একটি **Supervised Learning** algorithm
- এটি **Regression** সমস্যা সমাধান করে (continuous output)
- লক্ষ্য: দুটো variable-এর মধ্যে **linear relationship** খুঁজে বের করা

২. 📖 বিস্তারিত ব্যাখ্যা (Deep Explanation)

২.১ Simple Linear Regression (একটি feature)

কল্পনা করো একটি গ্রাফে অনেক বিন্দু (data points) ছড়িয়ে আছে। Linear Regression এই বিন্দুগুলো দিয়ে **সবচেয়ে ভালো সরলরেখা** টানে যা সব বিন্দুর কাছে থাকে।



এই রেখাটি হলো: $\hat{y} = w_0 + w_1 \times x$

২.২ Multiple Linear Regression (একাধিক feature)

যখন একাধিক input দিয়ে output predict করতে হয়:

উদাহরণ: বাড়ির দাম নির্ভর করে:

- x_1 = আয়তন (sqft)
- x_2 = বেডরুম সংখ্যা
- x_3 = লোকেশন score

$$\hat{y} = w_0 + w_1x_1 + w_2x_2 + w_3x_3$$

২.৩ ভেতরে কীভাবে কাজ করে?

ধাপ ১: Random weights (w) দিয়ে শুরু করো

ধাপ ২: প্রতিটি data point-এর জন্য prediction করো

ধাপ ৩: Actual vs Predicted-এর পার্থক্য (error) হিসাব করো

ধাপ ৪: Error কমাতে weights আপডেট করো

ধাপ ৫: যতক্ষণ না error সবচেয়ে কম হয়, ততক্ষণ ৩-৪ ধাপ repeat করো

২.৪ গুরুত্বপূর্ণ Terms

Term	বাংলা অর্থ	ব্যাখ্যা
Coefficient (w)	সহগ	প্রতিটি feature কতটুকু গুরুত্ব পাবে
Intercept (w_0)	ছেদ বিন্দু	$X=0$ হলে Y কত হবে
Residual	অবশিষ্ট ত্রুটি	Actual - Predicted মান
Slope	ঢাল	রেখার কতটা খাড়া
Overfitting	অতিরিক্ত ফিটিং	Training data মুখস্থ করে ফেলা
Underfitting	অপর্যাপ্ত ফিটিং	Pattern বুঝতে না পারা

৩. Math / Theory

৩.১ মূল সমীকরণ

Simple Linear Regression:

$$\hat{y} = w_0 + w_1 \cdot x$$

Multiple Linear Regression:

$$\hat{y} = w_0 + w_1x_1 + w_2x_2 + \dots + w_nx_n$$

Matrix Form (Vector Notation):

$$\hat{y} = Xw$$

যেখানে:

- $\hat{y}$ (y-hat) = predicted value (আমাদের অনুমান)
- w_0 = bias/intercept (রেখা কোথায় শুরু হয়)
- $w_1, w_2, \dots$ = weights/coefficients (feature-এর প্রভাব)
- $x_1, x_2, \dots$ = input features (input মান)
- X = Feature matrix
- w = Weight vector

৩.২ Cost Function (MSE)

আমরা কতটা ভুল করছি তা পরিমাপ করার জন্য **Mean Squared Error (MSE)** ব্যবহার করা হয়:

$$MSE = (1/n) \times \sum (y_i - \hat{y}_i)^2$$

অথবা **Sum of Squared Errors (SSE)**:

$$SSE = \sum (y_i - \hat{y}_i)^2 = \sum (y_i - w_0 - w_1x_i)^2$$

যেখানে:

- n = data point-এর সংখ্যা
- y_i = actual (আসল) মান
- $\hat{y}_i$ = predicted (অনুমানিত) মান
- $\sum$ = সব data point-এর যোগফল

কেন Square করি?

- (+) positive এবং (-) negative error বাতিল হয়ে না যায়

- বড় error-কে বেশি penalty দেওয়া হয়

৩.৩ Ordinary Least Squares (OLS) — Closed Form Solution

সরাসরি গণিত দিয়ে সেরা weight বের করা যায়:

$$w = (X^T X)^{-1} X^T y$$

Simple Linear Regression-এর জন্য:

$$w_1 = \frac{\sum [(x_i - \bar{x})(y_i - \bar{y})]}{\sum [(x_i - \bar{x})^2]}$$

$$w_0 = \bar{y} - w_1 \cdot \bar{x}$$

যেখানে:

- $\bar{x}$ = x-এর গড় (mean)
- $\bar{y}$ = y-এর গড় (mean)

৩.৪ Gradient Descent

যখন dataset বিশাল হয়, OLS computationally ব্যয়বহুল। তখন **Gradient Descent** ব্যবহার করা হয়:

Update Rule:

$$w := w - \alpha \cdot \partial \text{MSE} / \partial w$$

Partial Derivatives:

$$\partial \text{MSE} / \partial w_0 = (-2/n) \times \sum (y_i - \hat{y}_i)$$

$$\partial \text{MSE} / \partial w_1 = (-2/n) \times \sum (y_i - \hat{y}_i) \cdot x_i$$

যেখানে:

- α (alpha) = learning rate (কত বড় পদক্ষেপ নেব)
- $:=$ = update/assign করা হচ্ছে

৩.৫ Manual Calculation উদাহরণ

Data:

| x (পড়ার ঘণ্টা) | y (নম্বর) |

|-----|-----|

| 1 | 40 |

| 2 | 50 |

| 3 | 60 |

| 4 | 80 |

| 5 | 90 |

Step 1: গড় বের করো

$$\bar{x} = (1+2+3+4+5) / 5 = 3$$

$$\bar{y} = (40+50+60+80+90) / 5 = 64$$

Step 2: w_1 হিসাব করো

$$\begin{aligned}\sum [(x_i - \bar{x})(y_i - \bar{y})] &= (1-3)(40-64) + (2-3)(50-64) + (3-3)(60-64) + (4-3)(80-64) \\ &= (-2)(-24) + (-1)(-14) + (0)(-4) + (1)(16) + (2)(26) \\ &= 48 + 14 + 0 + 16 + 52 = 130\end{aligned}$$

$$\sum [(x_i - \bar{x})^2] = (-2)^2 + (-1)^2 + 0^2 + 1^2 + 2^2 = 4+1+0+1+4 = 10$$

$$w_1 = 130 / 10 = 13$$

Step 3: w_0 হিসাব করো

$$w_0 = \bar{y} - w_1 \cdot \bar{x} = 64 - 13 \times 3 = 64 - 39 = 25$$

ফলাফল: $\hat{y} = 25 + 13x$

Prediction: যদি কেউ ৬ ঘণ্টা পড়ে → $\hat{y} = 25 + 13 \times 6 = 103$ নম্বর (100 এর বেশি, তাই বাস্তবে limit আছে)

৩.৬ Evaluation Metrics

R² Score (Coefficient of Determination):

$$R^2 = 1 - (SS_{res} / SS_{tot})$$

$$SS_{res} = \sum (y_i - \hat{y}_i)^2 \quad \leftarrow \text{আমাদের model-এর error}$$

$$SS_{tot} = \sum (y_i - \bar{y})^2 \quad \leftarrow \text{শুধু mean দিয়ে করলে error কত হতো}$$

- $R^2 = 1 \rightarrow$ perfect prediction
- $R^2 = 0 \rightarrow$ model কিছুই শিখেনি

RMSE (Root Mean Squared Error):

$$RMSE = \sqrt{MSE} = \sqrt{[(1/n) \times \sum (y_i - \hat{y}_i)^2]}$$

MAE (Mean Absolute Error):

$$MAE = (1/n) \times \sum |y_i - \hat{y}_i|$$

8. Code Example (Python)

```
# =====  
# LINEAR REGRESSION - সম্পূর্ণ Python উদাহরণ  
# =====  
  
import numpy as np  
import matplotlib.pyplot as plt  
from sklearn.linear_model import LinearRegression  
from sklearn.model_selection import train_test_split  
from sklearn.metrics import mean_squared_error, r2_score  
  
# -----  
# ধাপ ১: ডেটা তৈরি করো  
# -----  
  
# random seed ঠিক করো যাতে প্রতিবার একই ফলাফল পাই  
np.random.seed(42)  
  
# পড়ার ঘণ্টা (1 থেকে 10)  
hours_studied = np.random.uniform(1, 10, 100) # 100 টি student  
  
# নম্বর = 10 * ঘণ্টা + 20 + কিছু noise  
marks = 10 * hours_studied + 20 + np.random.normal(0, 5, 100)  
  
# reshape করতে হবে কারণ sklearn 2D array চায়  
X = hours_studied.reshape(-1, 1) # shape: (100, 1)  
y = marks # shape: (100,)  
  
print("Data তৈরি হয়েছে!")  
print(f"X shape: {X.shape}, y shape: {y.shape}")  
  
# -----  
# ধাপ ২: Train-Test Split করো  
# -----  
  
# 80% training, 20% testing  
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.2, random_state=42  
)  
  
print(f"\nTraining samples: {len(X_train)}")  
print(f"Testing samples: {len(X_test)}")
```

```

# -----
# ধাপ ৩: Model তৈরি ও Train করো
# -----

# Linear Regression model তৈরি করো
model = LinearRegression()

# Model train করো (fit = শেখানো)
model.fit(X_train, y_train)

# শেখা parameters দেখো
print(f"\n📊 Model Parameters:")
print(f"Intercept (w0): {model.intercept_:.2f}") # bias
print(f"Slope (w1): {model.coef_[0]:.2f}") # weight

# -----
# ধাপ ৪: Prediction করো
# -----

# Test data-তে prediction করো
y_pred = model.predict(X_test)

# নতুন student: ৭ ঘণ্টা পড়লে কত নম্বর পাবে?
new_hours = np.array([[7]])
predicted_marks = model.predict(new_hours)
print(f"\n🎯 নতুন prediction:")
print(f"৭ ঘণ্টা পড়লে নম্বর হবে: {predicted_marks[0]:.1f}")

# -----
# ধাপ ৫: Model Evaluate করো
# -----

mse = mean_squared_error(y_test, y_pred) # MSE হিসাব করো
rmse = np.sqrt(mse) # RMSE হিসাব করো
r2 = r2_score(y_test, y_pred) # R2 score হিসাব করো

print(f"\n📈 Model Performance:")
print(f"MSE : {mse:.2f}") # গড় বর্গ ত্রুটি
print(f"RMSE : {rmse:.2f}") # মূল গড় বর্গ ত্রুটি
print(f"R2 : {r2:.4f}") # কতটুকু variance explain করতে

```

```
# -----  
# ধাপ ৬: Visualization করো  
# -----  
  
plt.figure(figsize=(10, 6))  
  
# Training data scatter plot (নীল বিন্দু)  
plt.scatter(X_train, y_train, color='blue', alpha=0.5, label='Training Data')  
  
# Test data scatter plot (সবুজ বিন্দু)  
plt.scatter(X_test, y_test, color='green', alpha=0.7, label='Test Data')  
  
# Regression line (লাল রেখা)  
x_line = np.linspace(1, 10, 100).reshape(-1, 1)  
y_line = model.predict(x_line)  
plt.plot(x_line, y_line, color='red', linewidth=2, label=f'Regression Line')  
  
# গ্রাফ সাজাও  
plt.xlabel('পড়ার ঘণ্টা', fontsize=12)  
plt.ylabel('পরীক্ষার নম্বর', fontsize=12)  
plt.title('Linear Regression: পড়ার ঘণ্টা vs নম্বর', fontsize=14)  
plt.legend()  
plt.grid(True, alpha=0.3)  
plt.tight_layout()  
plt.savefig('linear_regression_plot.png', dpi=150)  
plt.show()  
print("\nGraph সেভ হয়েছে!")
```

Expected Output:

Data তৈরি হয়েছে!

X shape: (100, 1), y shape: (100,)

Training samples: 80

Testing samples: 20



Model Parameters:

Intercept (w_0): 20.45

Slope (w_1): 9.98



নতুন prediction:

৭ ঘণ্টা পড়লে নম্বর হবে: 90.3



Model Performance:

MSE : 26.87

RMSE : 5.18

R^2 : 0.9541

Graph সেভ হয়েছে!

Scratch থেকে (NumPy দিয়ে) Linear Regression:

```

# =====
# SCRATCH থেকে LINEAR REGRESSION (কোনো library ছাড়া)
# =====

import numpy as np

class LinearRegressionScratch:
    """Scratch থেকে তৈরি Linear Regression"""

    def __init__(self, learning_rate=0.01, epochs=1000):
        self.lr = learning_rate          # কতটুকু পদক্ষেপ নেব
        self.epochs = epochs              # কতবার update করব
        self.weights = None               # feature weights
        self.bias = None                  # bias term

    def fit(self, X, y):
        """Model train করো (OLS বা Gradient Descent)"""
        n_samples = X.shape[0]           # কতগুলো data আছে
        n_features = X.shape[1]           # কতগুলো feature আছে

        # ১. Random weights দিয়ে শুরু করো
        self.weights = np.zeros(n_features)
        self.bias = 0
        self.loss_history = []             # প্রতি epoch-এর loss সংরক্ষণ

        # ২. Gradient Descent loop
        for epoch in range(self.epochs):
            # ২.১ Forward pass: prediction করো
            y_pred = np.dot(X, self.weights) + self.bias  #  $\hat{y} = Xw + b$ 

            # ২.২ Error হিসাব করো
            error = y_pred - y             # কতটুকু ভুল হলো

            # ২.৩ Gradients হিসাব করো (derivative)
            dw = (2/n_samples) * np.dot(X.T, error)      #  $\partial \text{MSE} / \partial w$ 
            db = (2/n_samples) * np.sum(error)             #  $\partial \text{MSE} / \partial b$ 

            # ২.৪ Weights আপডেট করো
            self.weights -= self.lr * dw          #  $w = w - \alpha \cdot \nabla w$ 
            self.bias -= self.lr * db             #  $b = b - \alpha \cdot \nabla b$ 

```

```

        # ২.৫ MSE হিসাব করো
        mse = np.mean(error ** 2)
        self.loss_history.append(mse)

        # প্রতি ২০০ epoch-এ progress দেখাও
        if epoch % 200 == 0:
            print(f"Epoch {epoch}: MSE = {mse:.4f}")

    return self

def predict(self, X):
    """নতুন data-র জন্য prediction করো"""
    return np.dot(X, self.weights) + self.bias

# ডেটা
np.random.seed(0)
X = np.random.rand(100, 1) * 10          # 1 টি feature
y = 3.5 * X.flatten() + 7 + np.random.randn(100) * 2 # true: w=3.5, b=7

# Model train করো
model = LinearRegressionScratch(learning_rate=0.005, epochs=1000)
model.fit(X, y)

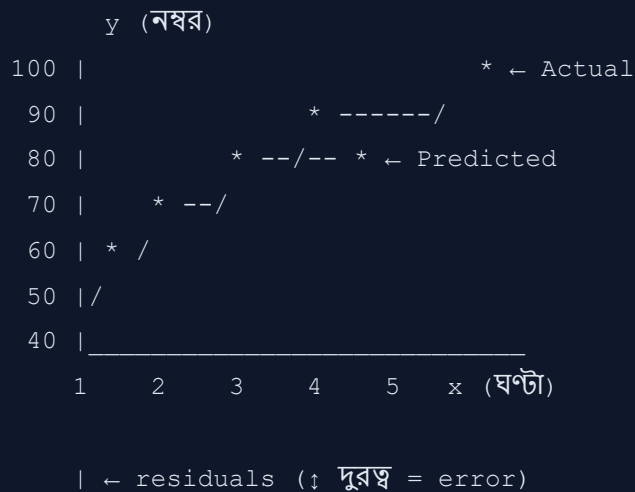
# Result দেখো
print(f"\nশেখা Weight: {model.weights[0]:.4f} (আসল: 3.5)")
print(f"শেখা Bias: {model.bias:.4f} (আসল: 7.0)")

# Prediction
new_x = np.array([[6.5]])
pred = model.predict(new_x)
print(f"\n6.5 input হলে output: {pred[0]:.2f}")
print(f"Expected output: {3.5*6.5 + 7:.2f}")

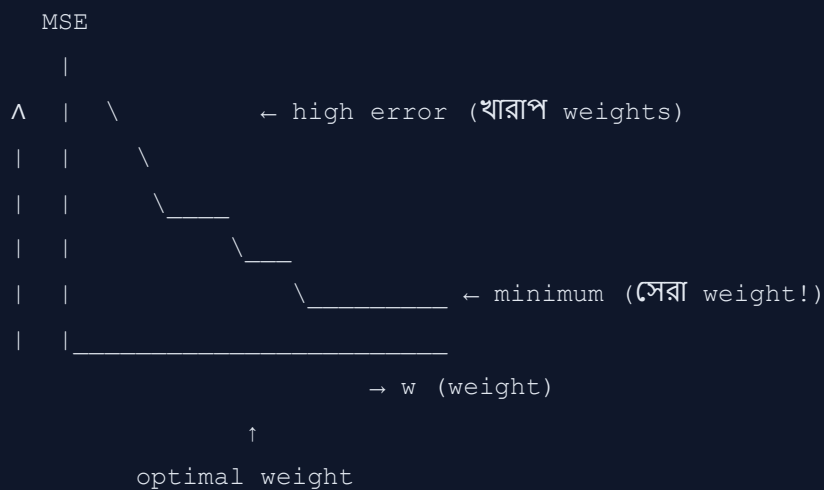
```

৫. 🎨 Visual / Diagram

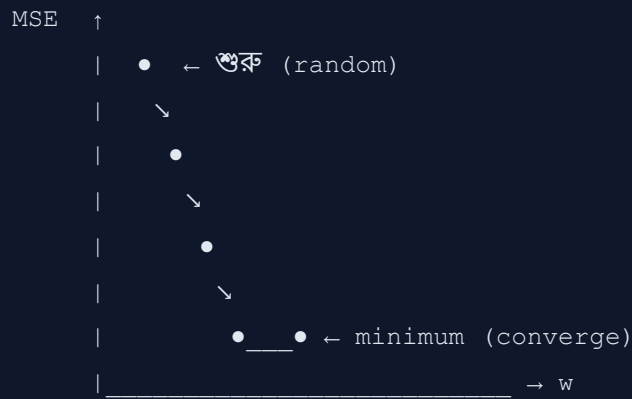
৫.১ Linear Regression-এর মূল ধারণা



৫.২ Cost Function (MSE) Landscape



৫.৩ Gradient Descent পদক্ষেপ



প্রতিটি • = একটি Epoch

↘ = gradient-এর বিপরীত দিকে যাওয়া

৫.৪ Simple vs Multiple Linear Regression

Simple (1 feature):

y

| /

| / ← regression line

| /

| /

| /

| /

| / _____ x

Multiple (2 feature):

y

| \

| \ ← regression plane

| \

----- x₂

x₁

৫.৫ Underfitting vs Good Fit vs Overfitting

Underfitting
(খুব সরল)

* * *

* *

* *

ভুল খুব বেশি
Test data-য়
ও ভুল বেশি

Good Fit
(সঠিক)

* * *

* / * \ *

* *

ভুল মাঝামাঝি
Test data-য়
ভুল কম

Overfitting
(অতিরিক্ত জটিল)

*

* ---*---*

* * * *

Training এ ১০০%
Test data-য় ভুল
বেশি

৬. Real-world Use Cases

১. Real Estate (বাড়ির দাম নির্ধারণ)

Company: Zillow, 99acres, Bproperty

- Input: আয়তন, বেডরুম, লোকেশন, বয়স
- Output: বাড়ির দাম
- বাস্তব প্রভাব: Zillow-এর "Zestimate" feature কোটি কোটি ডলারের সম্পদ মূল্যায়ন করে

২. Stock Price Prediction (শেয়ার বাজার)

Company: Goldman Sachs, Renaissance Technologies

- Input: historical prices, volume, market indicators
- Output: ভবিষ্যৎ মূল্য
- সতর্কতা: stock market non-linear, তাই single linear model যথেষ্ট নয়

৩. Healthcare (স্বাস্থ্য খাত)

Use: FDA Drug Trials

- Input: ওষুধের ডোজ, রোগীর বয়স, ওজন
- Output: ব্লাড প্রেশার কমানোর পরিমাণ
- গুরুত্ব: জীবন বাঁচানোর সিদ্ধান্তে ব্যবহার হয়

৪. Agriculture (কৃষি)

Use: Crop Yield Prediction

- Input: বৃষ্টিপাত, তাপমাত্রা, সার ব্যবহার
- Output: ফসলের পরিমাণ (টন/হেক্টর)
- বাংলাদেশ সহ অনেক দেশের কৃষি মন্ত্রণালয় ব্যবহার করে

৫. E-commerce (অনলাইন কেনাকাটা)

Company: Amazon, Daraz

- Input: advertisement spend, seasonality, competitor pricing

- Output: বিক্রির পরিমাণ
- Linear Regression sales forecasting-এর baseline model হিসাবে ব্যবহৃত হয়

৭. Pros & Cons

সুবিধা 	অসুবিধা 
বোঝা ও ব্যাখ্যা করা সহজ	শুধু linear relationship ধরতে পারে
Train করা দ্রুত ও সহজ	Outlier-এর প্রতি অত্যন্ত sensitive
Interpretable — প্রতিটি coefficient-এর অর্থ আছে	Non-linear data-তে কাজ করে না
কম data-তেও কাজ করে	Multicollinearity সমস্যা তৈরি করে
অন্য ML model-এর baseline হিসেবে ব্যবহারযোগ্য	Feature scaling প্রয়োজন হয়
Regularization (Ridge/Lasso) সহজে যোগ করা যায়	Complex patterns ধরতে পারে না
High-dimensional space-এ OLS কাজ করে	Assumptions অনেক (normality, homoscedasticity)

৮. Common Mistakes & Gotchas

ভুল ১: Feature Scaling না করা 

```
# ভুল: feature-এর scale আলাদা হলে gradient descent ঠিকমতো কাজ করে না
X = [[1000, 2], [1500, 3], [2000, 4]] # sqft and bedrooms – আলাদা scale!

# সঠিক: StandardScaler ব্যবহার করো
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

ভুল ২: Outlier ignore করা ❌

আসল ডেটা:	Outlier সহ ডেটা:
<pre> * * * * /← সঠিক line * / * / </pre>	<pre> * * ← এই outlier * ← regression line টেনে নামাবে * ____ ← ভুল line * * ← ভুল direction </pre>

Solution: IQR method বা Z-score দিয়ে outlier সরাও।

ভুল ৩: Train-Test Split না করা ❌

```
# ভুল: পুরো data দিয়ে train ও test করা
model.fit(X, y)
score = model.score(X, y) # ← ভুল! Overfitting দেখাবে না

# সঠিক:
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
model.fit(X_train, y_train)
score = model.score(X_test, y_test) # ← সঠিক!
```

ভুল ৪: Categorical Variable সরাসরি ব্যবহার করা ❌

```
# ভুল: "ঢাকা", "চট্টগ্রাম", "সিলেট" সরাসরি ব্যবহার
X['city'] = ['ঢাকা', 'চট্টগ্রাম', 'ঢাকা'] # ❌

# সঠিক: One-Hot Encoding করো
X_encoded = pd.get_dummies(X, columns=['city']) # ✅
```

ভুল ৫: R^2 একাই দেখে সিদ্ধান্ত নেওয়া ❌

- $R^2 = 0.95$ মানেই model ভালো নয়!
- Training R^2 বেশি কিন্তু Test R^2 কম $\rightarrow$ Overfitting
- সবসময় **Training + Test উভয় metric** দেখো

ভুল ৬: Assumptions Check না করা ❌

Linear Regression-এর assumptions আছে। এগুলো verify করো:

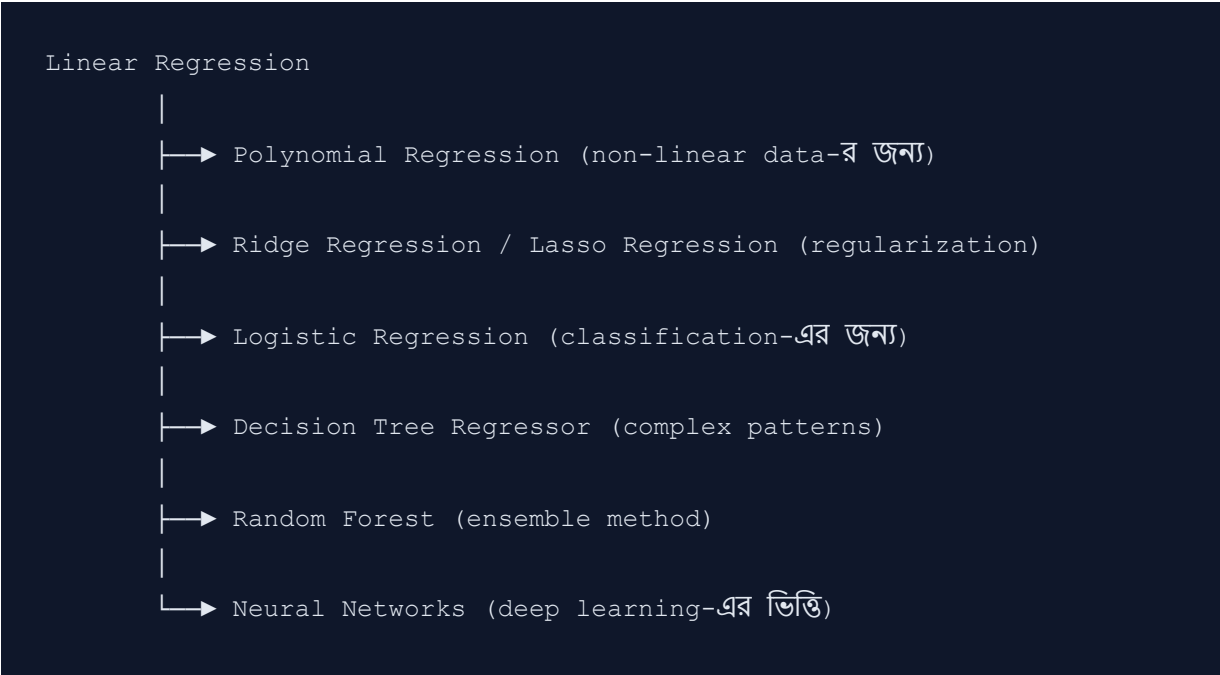
- ✅ Data linearly related কিনা (scatter plot দেখো)
- ✅ Residuals normally distributed কিনা (QQ plot)
- ✅ Homoscedasticity আছে কিনা (residual plot)
- ✅ Multicollinearity নেই কিনা (VIF check)

৯. 🔗 Related Topics

আগে যা জানা দরকার (Prerequisites):

- ➡ **Statistics Basics:** Mean, Variance, Standard Deviation
- ➡ **Linear Algebra:** Matrix multiplication, inverse
- ➡ **Calculus:** Derivative, Partial derivative
- ➡ **Python/NumPy:** Array operations
- ➡ **Supervised Learning:** কী এবং কেন

এরপর কী শেখা উচিত (Next Steps):



Regularization Techniques:

Method	কী করে	কখন ব্যবহার
Ridge (L2)	বড় weights কমায়	Multicollinearity থাকলে
Lasso (L1)	কিছু weights শূন্য করে	Feature selection দরকার হলে
ElasticNet	L1+L2 একসাথে	উভয় সুবিধা দরকার হলে

১০. 🧠 Memory Tricks

মনে রাখার সহজ কৌশল

🔑 "LIME" ট্রিক:

- **L** = Linear relationship খোঁজে
- **I** = Input features দিয়ে
- **M** = MSE কমায়
- **E** = Error (residual) minimize করে

📐 Slope মনে রাখো:

"Rise over Run" → ওপরে উঠুন, তারপর সামনে যান

Rise (y-এর পরিবর্তন)

$$w_1 = \frac{\text{Rise}}{\text{Run}}$$

Run (x-এর পরিবর্তন)

🎯 একটি লাইনে সারসংক্ষেপ:

"Linear Regression হলো ডেটার মধ্যে দিয়ে সবচেয়ে ভালো সরলরেখা টানার শিল্প — যা দিয়ে ভবিষ্যৎ অনুমান করা যায়।"

💡 সহজ উপমা:

- Linear Regression = রুলার দিয়ে দাগ টানা
- Gradient Descent = পাহাড় থেকে সবচেয়ে দ্রুত নামার পথ খোঁজা
- MSE = গড় ভুলের পরিমাণ
- R^2 = "আমার model কতটুকু ভালো?" (0 থেকে 1)

📦 Algorithm সারসংক্ষেপ:

```
১. Data নাও
    ↓
২.  $w_0, w_1$  random দাও
    ↓
৩.  $\hat{y} = w_0 + w_1x$  হিসাব করো
    ↓
৪.  $MSE = \sum (y - \hat{y})^2 / n$  হিসাব করো
    ↓
৫. Gradient হিসাব ও weights আপডেট করো
    ↓
৬. MSE আর কমছে না? → Stop!
    ↓
৭. Final model ready! 🎉
```



References

- [Scikit-learn Documentation](#)
- [StatQuest with Josh Starmer — Linear Regression](#)
- [Andrew Ng — Machine Learning Course \(Coursera\)](#)
- [Wikipedia: Linear Regression](#)
- [Analytics Vidhya: Linear Regression Guide](#)



তৈরি তারিখ: ২০২৬ |  পরবর্তী Review: ১ সপ্তাহ পর

Linear Regression — সম্পূর্ণ বাংলা নোট

তৈরি তারিখ: ২০২৬ | ML Notes Series | পরবর্তী নোট: Logistic Regression